

~/aws-iam-guide — zsh

\$ whoami --role CloudSecurityEngineer

AWS Technical Documentation Series · Cloud Architecture Reference

AWS IAM



Identity & Access Management

A Complete, Practical Reference Guide for DevOps & Cloud Engineers

// TOPICS COVERED IN THIS GUIDE

- | | | | |
|----|---------------------------|----|----------------------------|
| 01 | Root Account vs IAM User | 06 | Policy Types — 3 Kinds |
| 02 | IAM Users, Groups & Roles | 07 | Policy Evaluation Logic |
| 03 | IAM Policy JSON Structure | 08 | Managed vs Inline Policies |
| 04 | Effect, Action & Resource | 09 | IAM Best Practices |
| 05 | Condition Operators | 10 | Real-World Policy Examples |

20

PAGES

13

CHAPTERS

5

DIAGRAMS

10

POLICY EXAMPLES

CHAPTER

Table of Contents

Chapter 01	Introduction to AWS IAM	3
Chapter 02	Root Account vs IAM User	4
Chapter 03	IAM Core Entities	5
Chapter 04	What is an IAM Policy?	6
Chapter 05	Policy JSON Structure Deep Dive	7
Chapter 06	Effect, Action, Resource & Condition	8
Chapter 07	Types of IAM Policies	9
Chapter 08	Policy Evaluation Logic	10
Chapter 09	IAM Groups & Roles	11
Chapter 10	Managed vs Inline Policies	12
Chapter 11	IAM Best Practices	13
Chapter 12	Common Policy Examples	14
Chapter 13	Summary & Quick Reference	15

CHAPTER 01

Introduction to AWS IAM

AWS Identity and Access Management is the backbone of security in any AWS environment.

AWS IAM (Identity and Access Management) is a global AWS service that enables you to manage access to AWS services and resources securely. IAM answers the fundamental security question: **Who can do What to Which resources, and under What conditions?**

Why IAM Matters

- IAM is **free of charge** — there is no additional cost for using IAM.
- IAM is **global** — once created, users and policies apply across all AWS regions.
- IAM provides **fine-grained access control** down to individual API calls.
- Without IAM, **every user would have full root-level access** — a critical security risk.
- IAM integrates with **virtually every AWS service** — S3, EC2, RDS, Lambda, and more.

The Four Questions IAM Answers
Who (Principal) is making the request? · **What** (Action) do they want to do? · **Which** (Resource) AWS resource are they targeting? · **When / Where / How** (Condition) should this be allowed?

IAM Key Terminology

Term	Definition
Principal	An entity (user, role, application) that makes a request to AWS.
Authentication	Proving your identity (password, access key, MFA token).
Authorization	Determining what actions you are permitted to perform.
Policy	A JSON document that defines permissions — what is allowed or denied.
ARN	Amazon Resource Name — a unique identifier for every AWS resource.

CHAPTER 02

Root Account vs IAM User

Understanding the critical difference between the root account and IAM users.

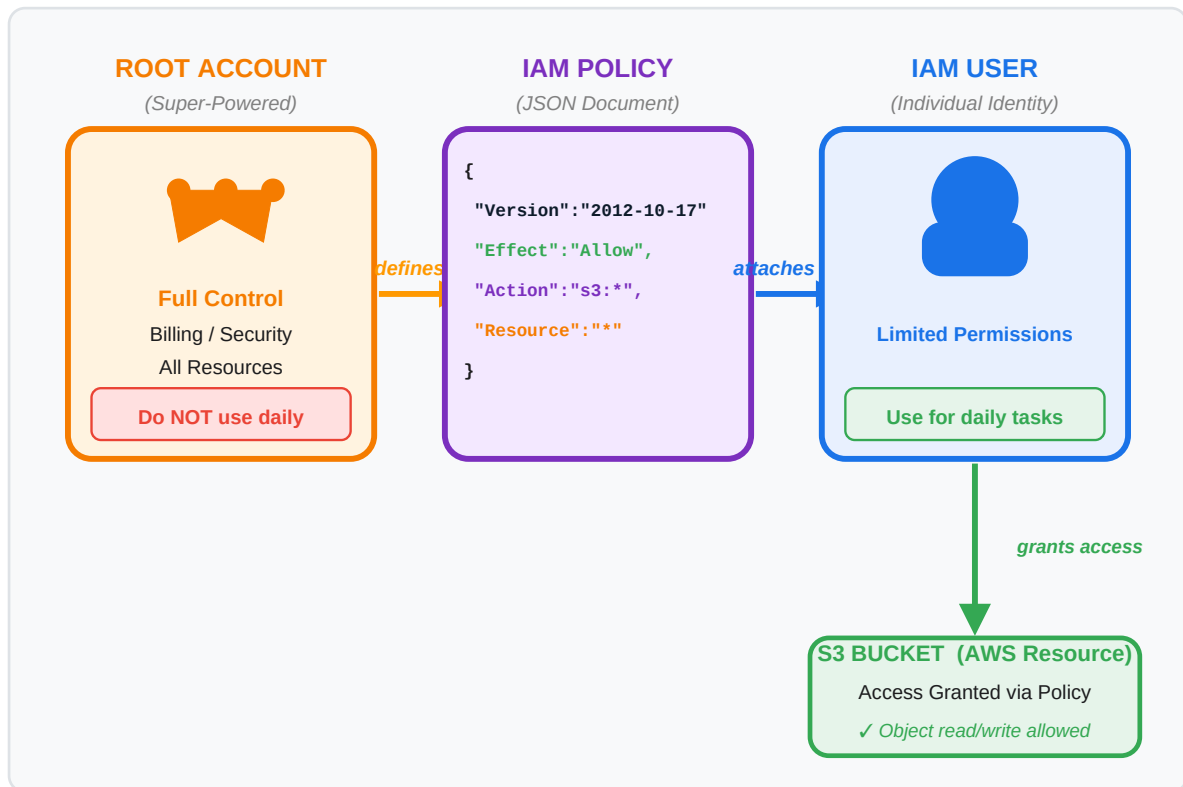


Figure 1: Root Account defines a Policy and attaches it to an IAM User, granting access to S3.

The Root Account

The root account is created when you first sign up for AWS. It uses the email address and password you provided during registration. This account has **unrestricted, absolute control** over every AWS service and resource in the account — including billing, security credentials, and the ability to close the entire account.

■ Root Account Warning

Never use the root account for day-to-day cloud management tasks. Create individual IAM users with the minimum required permissions instead. Enable MFA on the root account immediately after account creation and store the credentials securely.

The IAM User

An IAM User is an identity you create within your AWS account representing a person or application. By default, a newly created IAM user has **zero permissions** — they cannot access any AWS service until a policy explicitly grants them access. This principle of least privilege is fundamental to AWS security.

CHAPTER 03

IAM Core Entities

Users, Groups, Roles, and Policies — the four pillars of IAM.

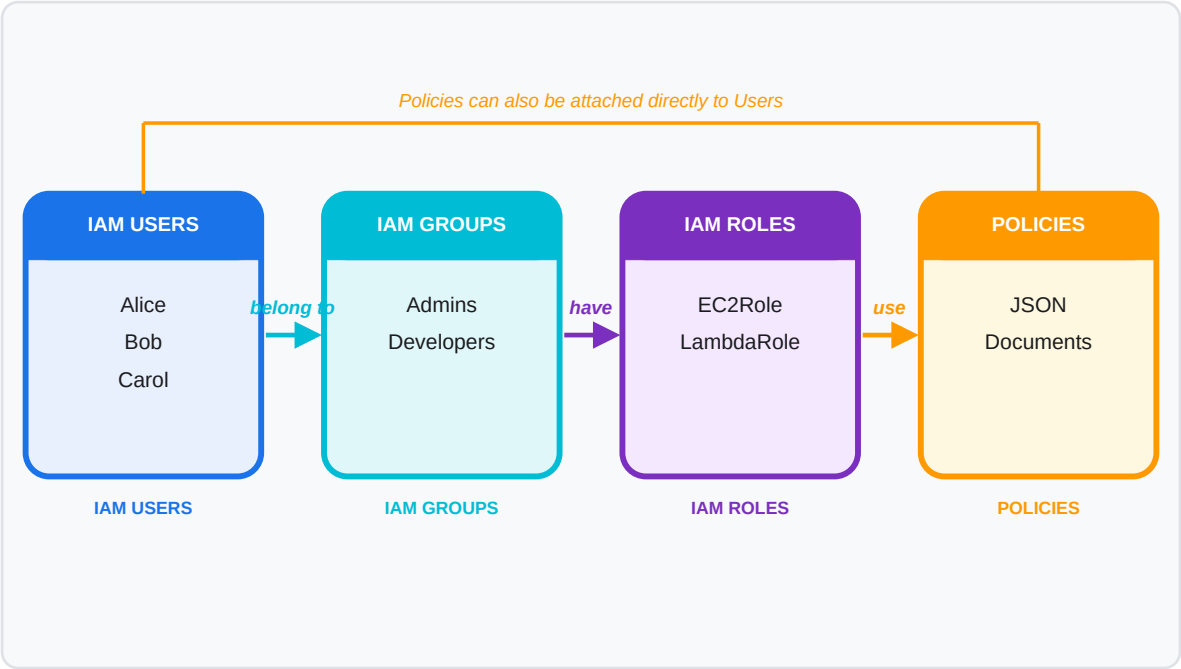


Figure 2: Relationship between IAM Users, Groups, Roles, and Policies.

The Four Core IAM Entities

Entity	Description	Use Case
IAM User	Represents a person or application with credentials.	Individual developer logins, CI/CD keys
IAM Group	Collection of IAM users sharing the same permissions.	Admins group, Developers group, Auditors group
IAM Role	Temporary identity assumed by users or AWS services.	EC2 instance accessing S3, Lambda execution
IAM Policy	JSON document defining allowed or denied actions.	Grants S3 read access, EC2 full access, etc.

CHAPTER 04

What is an IAM Policy?

Policies are the engine of IAM — they define permissions in precise JSON format.

An IAM Policy is a JSON (JavaScript Object Notation) document that formally specifies what AWS API actions are allowed or denied, on which resources, and under what conditions. Policies are the core mechanism through which access control is enforced in AWS.

How Policies Work

- You **write a policy** as a JSON document with precise permission statements.
- You **attach the policy** to a user, group, or role (the 'principal').
- When the principal makes an API call, AWS **evaluates the policy** to allow or deny.
- If **no policy explicitly allows** an action, it is **denied by default**.
- If **any policy explicitly denies** an action, it is denied — even if another allows it.

A Simple Example Policy

The policy below grants a user permission to list and read objects from all S3 buckets:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3ReadAccess",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": "*"
    }
  ]
}
```

Listing 1: IAM Policy granting S3 read/list access to all resources.

CHAPTER 05

Policy JSON Structure Deep Dive

Every element of an IAM policy JSON document, explained in detail.

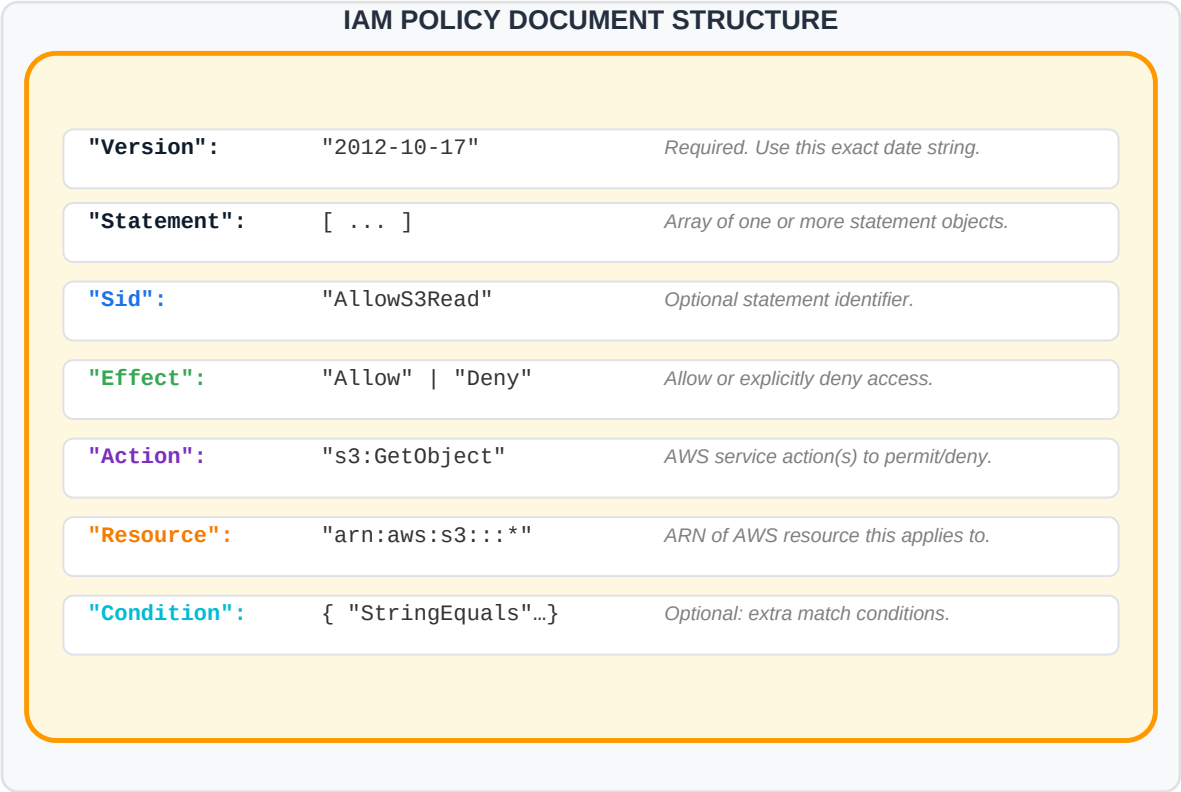


Figure 3: Anatomy of an IAM Policy JSON document — every key explained.

The Version Field

The **Version** field specifies the IAM policy language version. Always use **"2012-10-17"** — this is the current version and supports all policy features including policy variables like **`${aws:username}`**. The older version **"2008-10-17"** does not support policy variables and should not be used.

The Statement Array

The **Statement** field is an array containing one or more individual permission statements. Each statement is an independent unit that defines a specific permission. A single policy document can contain multiple statements, allowing you to combine different permissions (and even different effects) in one document.

CHAPTER 06

Effect, Action, Resource & Condition

The four statement-level keys that define precise permission boundaries.

Effect

The **Effect** key accepts exactly two values: **"Allow"** or **"Deny"**. This is the most impactful field in a policy statement. An explicit **Deny** always overrides any Allow, regardless of how many Allow statements exist elsewhere.

Action

The **Action** field specifies which AWS API actions the statement applies to. Actions follow the format **service:APIName** (e.g., **s3:GetObject**, **ec2:StartInstances**). You can use the ***** wildcard to match multiple actions:

- **"s3:*"** — all S3 actions
- **"s3:Get*"** — all S3 Get actions (GetObject, GetBucketPolicy, etc.)
- **"*"** — all actions on all services (use with extreme caution)

Resource

The **Resource** field identifies the specific AWS resources the statement applies to, using Amazon Resource Names (ARNs). ARNs follow the pattern:

```
arn:partition:service:region:account-id:resource
```

- **"arn:aws:s3:::my-bucket"** — a specific S3 bucket
- **"arn:aws:s3:::my-bucket/*"** — all objects in a specific S3 bucket
- **"arn:aws:ec2:us-east-1:123456789:instance/i-abc123"** — a specific EC2 instance
- **"*"** — all resources (often used for actions that have no specific resource)

Condition

The **Condition** element is optional and adds an extra layer of control. It lets you specify circumstances under which the statement is in effect. Common use cases:

- Allow access **only from specific IP address ranges** (e.g., your corporate network)
- Require **MFA authentication** before performing sensitive actions
- Restrict access to **specific times of day** or **specific AWS regions**

- Enforce **secure transport (HTTPS)** for all S3 API calls

CHAPTER 07

Types of IAM Policies

AWS offers three main policy types — each with distinct characteristics and use cases.

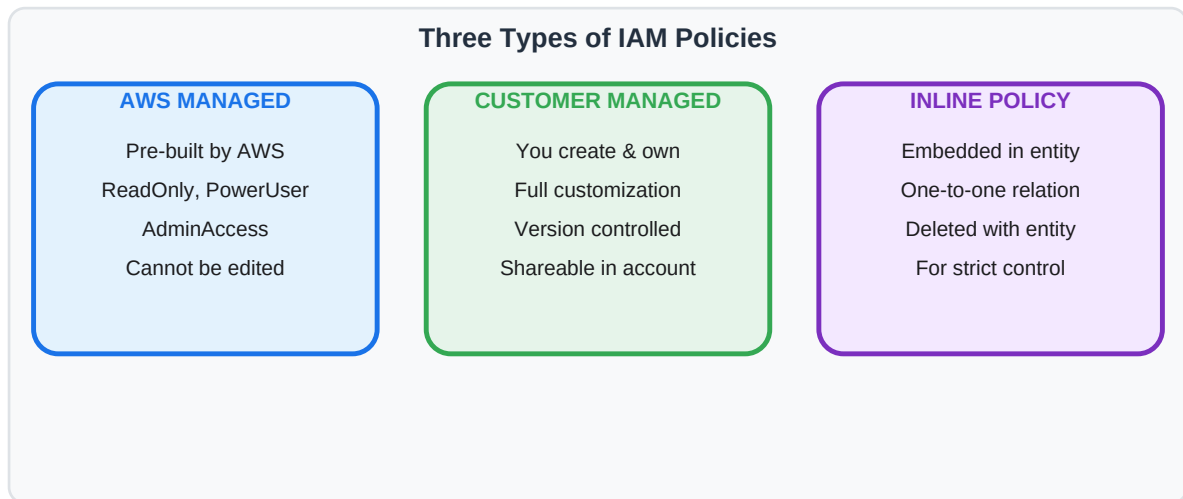


Figure 4: Three types of IAM Policies — AWS Managed, Customer Managed, and Inline.

AWS Managed Policies

AWS Managed Policies are pre-built, maintained, and updated by Amazon. They cover common use cases and follow AWS best practices. You cannot edit them, but you can attach them to any user, group, or role in your account. Examples include:

- **AdministratorAccess** — full access to all AWS services and resources
- **ReadOnlyAccess** — read-only access to all services
- **AmazonS3FullAccess** — full access to S3
- **AWSLambdaBasicExecutionRole** — permissions for Lambda to write logs

Customer Managed Policies

You create and manage these policies within your AWS account. They offer full customization and can be shared across multiple users, groups, and roles. AWS automatically maintains up to 5 versions of each policy, allowing you to roll back changes.

Inline Policies

Inline policies are embedded directly into a single IAM entity (user, group, or role). They are deleted when the entity is deleted. Use inline policies when you need a strict one-to-one relationship between a policy and an identity.

CHAPTER 08

Policy Evaluation Logic

How AWS decides whether to Allow or Deny a request — the evaluation order matters.

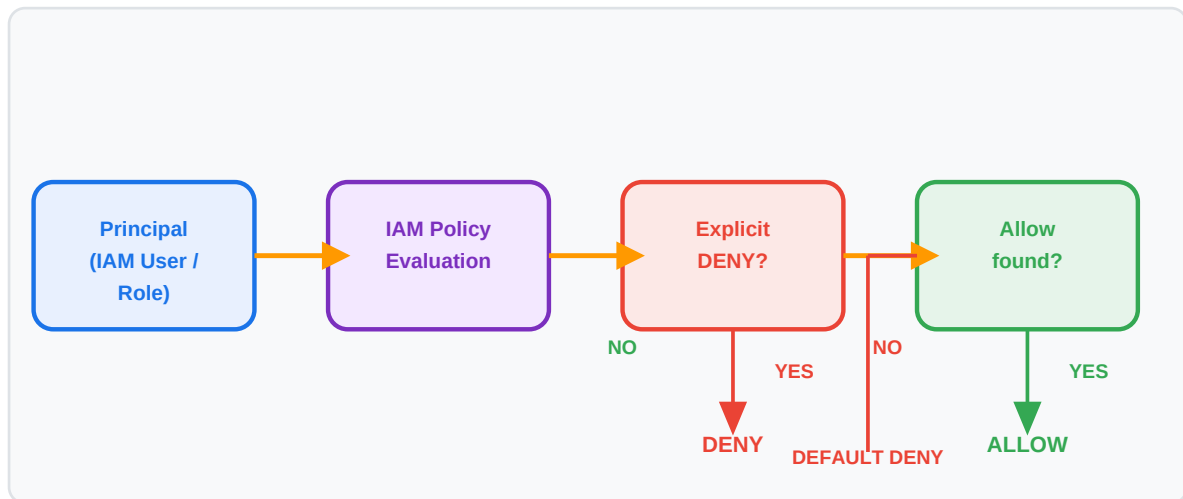


Figure 5: IAM Policy evaluation flow — explicit Deny always wins.

The Evaluation Order

1. Default Deny

Every request starts as implicitly denied. No access without explicit Allow.

2. Evaluate Policies

AWS collects all applicable policies (identity, resource, SCP, etc.).

3. Explicit Deny?

If any policy has an explicit Deny for the action, the request is denied immediately. Period.

4. Explicit Allow?

If any policy has an explicit Allow for the action (and no Deny), the request proceeds.

5. Final Default

If no explicit Allow exists, the request is denied by default.

Key Rule to Remember

Explicit DENY > Explicit ALLOW > Implicit DENY. An explicit Deny in any policy will always override an Allow in any other policy. This is the single most important concept in IAM policy evaluation.

CHAPTER 09

IAM Groups & Roles

Organizing users with Groups, and delegating access with Roles.

IAM Groups

An IAM Group is a collection of IAM users. Groups let you specify permissions for multiple users at once, making it easier to manage permissions for users with the same job function. Key points:

- Groups can contain multiple users, and a user can belong to multiple groups (up to 10).
- Groups cannot contain other groups — they are flat collections.
- Policies attached to a group apply to all users in that group.
- Common groups: Admins, Developers, Testers, Auditors, DBAs.

IAM Roles

An IAM Role is an identity with specific permissions that can be **assumed temporarily** by trusted entities. Unlike users, roles do not have long-term credentials (passwords or access keys). Instead, they provide temporary security credentials when assumed.

Roles are used in these key scenarios:

Scenario	Example
AWS Service Access	An EC2 instance needs to read from S3 — attach an IAM Role to the EC2 instance.
Cross-Account Access	Allow an IAM user in Account A to access resources in Account B.
Identity Federation	Allow corporate users (Active Directory) to access AWS without creating IAM users.
Application Access	Mobile/web apps need temporary AWS credentials — use Cognito with IAM Roles.

CHAPTER 10

Managed vs Inline Policies — Detailed Comparison

Choosing the right policy type for your use case.

Feature	AWS Managed	Customer Managed	Inline
Created by	Amazon Web Services	You (the account owner)	You (embedded)
Reusable	Yes — multi-entity	Yes — multi-entity	No — single entity
Editable	No	Yes — full control	Yes — direct
Versioning	AWS manages	Up to 5 versions	No versioning
Deleted with entity	No	No	Yes
Best for	Common permissions	Custom org policies	Strict 1-to-1 control
Visible in console	Yes — shared library	Yes — your policies	Only on the entity

Recommendation

For most use cases, prefer **Customer Managed Policies**. They give you full control and can be reused across multiple entities. Use **AWS Managed Policies** as a starting point and clone them when you need customization. Reserve **Inline Policies** only for strict one-to-one binding where you absolutely do not want the policy reused.

CHAPTER 11

IAM Best Practices

Security recommendations from AWS for managing IAM effectively.

✓ Lock Down the Root Account

Enable MFA on the root account immediately. Create a dedicated admin IAM user for daily tasks. Do not create access keys for the root account.

✓ Apply Least Privilege

Grant only the permissions required to perform a task. Start with minimal permissions and add more as needed. Regularly audit and remove unused permissions.

✓ Use Groups to Assign Permissions

Never assign permissions directly to individual users. Instead, create groups (e.g., Admins, Developers, Auditors) and assign permissions to groups.

✓ Enable MFA for Privileged Users

Require Multi-Factor Authentication for all IAM users with console access, especially those with administrative permissions.

✓ Use IAM Roles for Applications

Never embed access keys in application code or EC2 instances. Assign IAM Roles to EC2, Lambda, and other AWS services instead.

✓ Rotate Credentials Regularly

Establish a policy to rotate IAM user passwords and access keys every 90 days. Use IAM credential reports to identify old credentials.

✓ Use IAM Access Analyzer

Enable IAM Access Analyzer to identify resources shared with external entities and validate that your policies grant only the intended permissions.

CHAPTER 12

Common Policy Examples

Ready-to-use IAM policy patterns for real-world scenarios.

S3 Full Access to a Specific Bucket

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "s3:*",
    "Resource": [
      "arn:aws:s3:::my-bucket",
      "arn:aws:s3:::my-bucket/*"
    ]
  }]
}
```

Deny All Actions if MFA Not Present

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "*",
    "Resource": "*",
    "Condition": {
      "BoolIfExists": {
        "aws:MultiFactorAuthPresent": "false"
      }
    }
  }]
}
```

Allow EC2 Read-Only from Specific IP

```
{
```

```
"Version": "2012-10-17",
"Statement": [{
  "Effect": "Allow",
  "Action": "ec2:Describe*",
  "Resource": "*",
  "Condition": {
    "IpAddress": {
      "aws:SourceIp": "203.0.113.0/24"
    }
  }
}]
}
```

CHAPTER 13

Summary & Quick Reference

A concise reference card covering all key IAM concepts from this guide.

Concept	Key Takeaway
Root Account	Has absolute control. Enable MFA, never use for daily tasks.
IAM User	No permissions by default. Assign policies to grant access.
IAM Group	Collection of users. Assign permissions to groups, not individuals.
IAM Role	Temporary identity for services and cross-account access.
IAM Policy	JSON document with Version, Statement, Effect, Action, Resource, Condition.
Effect: Allow	Grants the specified actions on the specified resources.
Effect: Deny	Blocks actions. Explicit Deny overrides any Allow — always.
Action	AWS API call format: service:APIName — e.g., s3:GetObject.
Resource	ARN of the target resource — e.g., arn:aws:s3:::my-bucket/*.
Condition	Optional extra constraints: IP, MFA, time, region, tag values.
AWS Managed	Pre-built by AWS. Cannot edit. Easy to attach and share.
Customer Managed	You create and own. Fully customizable. Up to 5 versions.
Inline Policy	Embedded in entity. Deleted when entity is deleted.
Default Deny	If no policy explicitly allows, the action is denied.
Least Privilege	Grant only the minimum permissions needed for the task.

You're Ready to Use AWS IAM!
You now understand the complete IAM model — from the root account and IAM Users to JSON Policy anatomy, policy types, evaluation logic, groups, roles, and best practices. Start by enabling MFA on your root account, creating an admin IAM user, and always applying the principle of least privilege. Happy building on AWS!